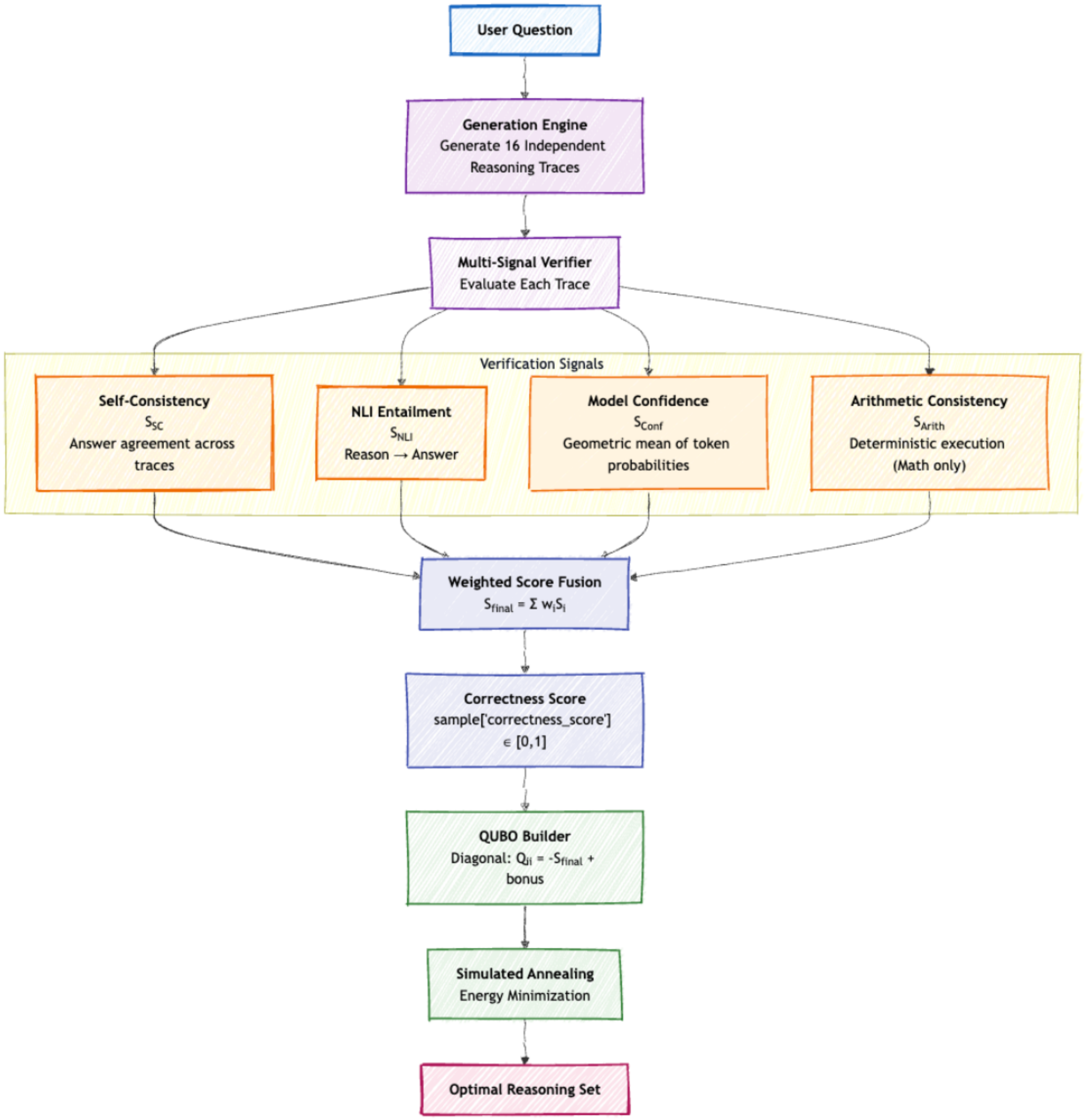
Prism Monthly Meet

QLoRA SFT + Hyperparameter Finetuning

20/08/2026

Recap



$$S_{\text{final}} = w_{\text{SC}} S_{\text{SC}} + w_{\text{NLI}} S_{\text{NLI}} + w_{\text{Conf}} S_{\text{Conf}} + w_{\text{Arith}} S_{\text{Arith}}$$

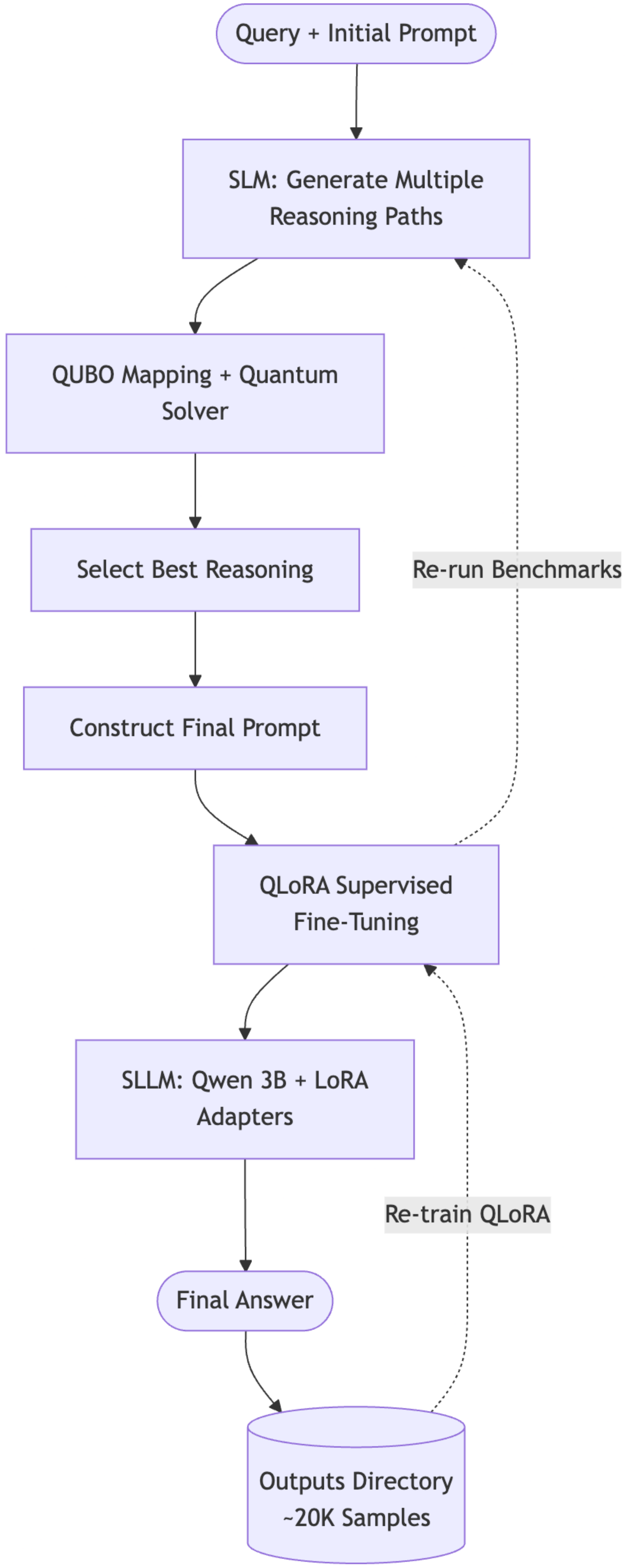
$$S_{\text{SC}} = \frac{\text{Frequency of Target Predicted Answer}}{\text{Total Number of Sampled Traces}}$$

$$\text{score} = \frac{e^{z_{\text{entail}}}}{e^{z_{\text{entail}}} + e^{z_{\text{contradict}}} + e^{z_{\text{neutral}}}}$$

$$S_{\text{Conf}} = \exp\left(\frac{1}{N}\sum_{i=1}^{N} \log P(t_i)\right)$$

$$S_{\text{Arith}} = \frac{\sum \text{Passed Assertions}}{\text{Total Assertions Identified}}$$

Task	wSC	wNLI	wConf	wArith
Math	0.4	0.3	0.2	0.1
Commonsense	0.5	0.3	0.2	0.0


Query + Initial Prompt
SLM: Generate Multiple Reasoning Paths
QUBO Mapping + Quantum Solver
Select Best Reasoning
Construct Final Prompt
QLoRA Supervised Fine-Tuning
SLLM: Qwen 3B + LoRA Adapters
Final Answer
Outputs Directory ~20K Samples
Re-run Benchmarks
Re-train QLoRA

Why QLoRa and not full fine tuning ?

Base Model: The core ~3B parameter base model stays completely frozen

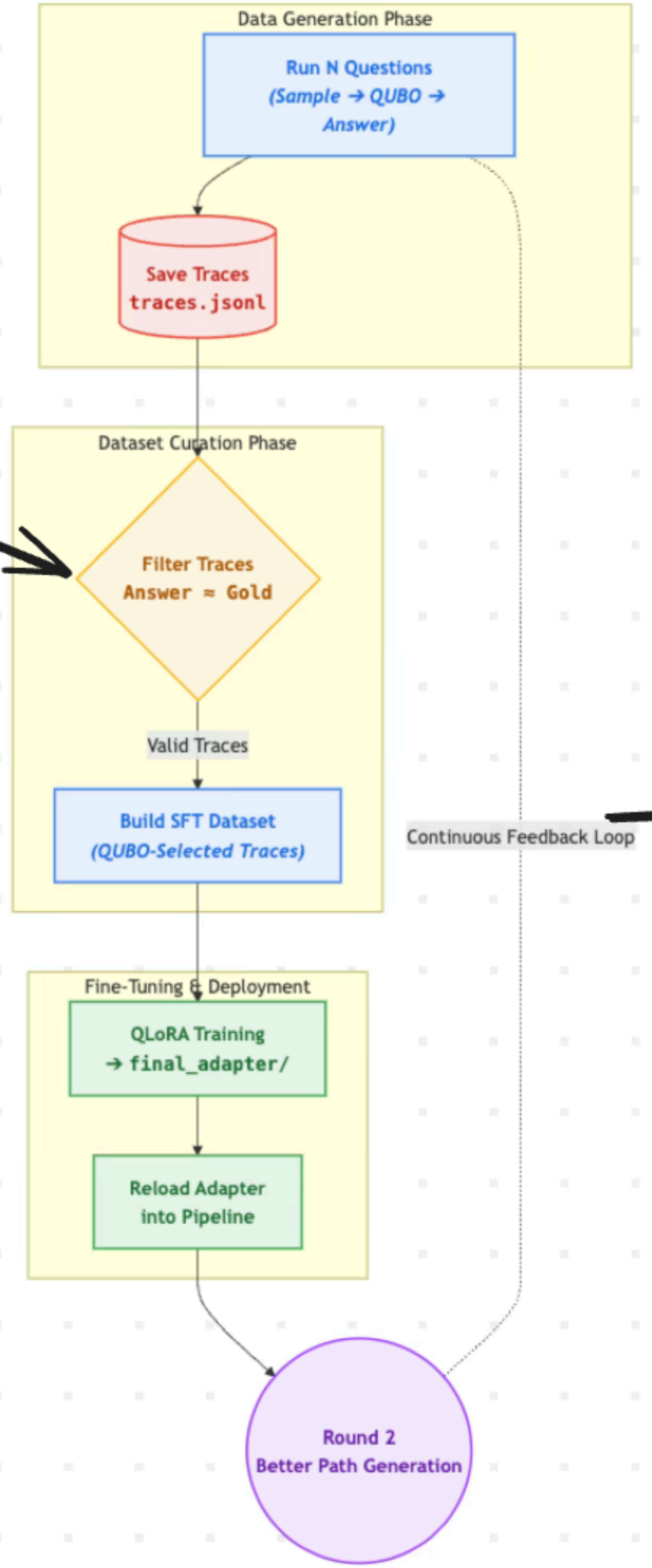
Minimal Parameter Training: Only ~0.1% of total parameters are trained by using lightweight LoRA (Low-Rank Adaptation) adapters.

Low Memory Footprint: Highly memory-efficient structure fits comfortably on an H100 GPU without risk of out-of-memory errors.

Compact Checkpoint Output: Generates small adapter files (~tens of MBs) instead of saving redundant, full ~3B parameter model weights.

Rapid Iteration: Facilitates fast, lightweight training cycles and quick iteration per fine-tuning round.

keep only those which mathces ground truth



Round	Model	Action
Round 1	Base Qwen 2.5-3B	Run benchmark → collect traces → QLoRA
Round 2	Base + LoRA (round 1)	Re-run same questions → new traces → QLoRA again
Round 3	Base + LoRA (round 2)	Repeat (config: `iterative_rounds: 3`)

Model and Quantization

Setting	Value
Base model	Qwen/Qwen2.5-3B-Instruct
Quantization	4-bit NF4 (bitsandbytes)
Double quant	Yes
Compute dtype	bfloat16 on H100 (fp16 fallback)
Attention	SDPA
Base weights	Frozen
Trainable	LoRA adapters only
Gradient checkpointing	On (saves memory)

Inference v/s training

Phase	**Quantization**
Pipeline inference (sampling + answer)	FP16, no 4-bit
QLoRA training	4-bit base + LoRA

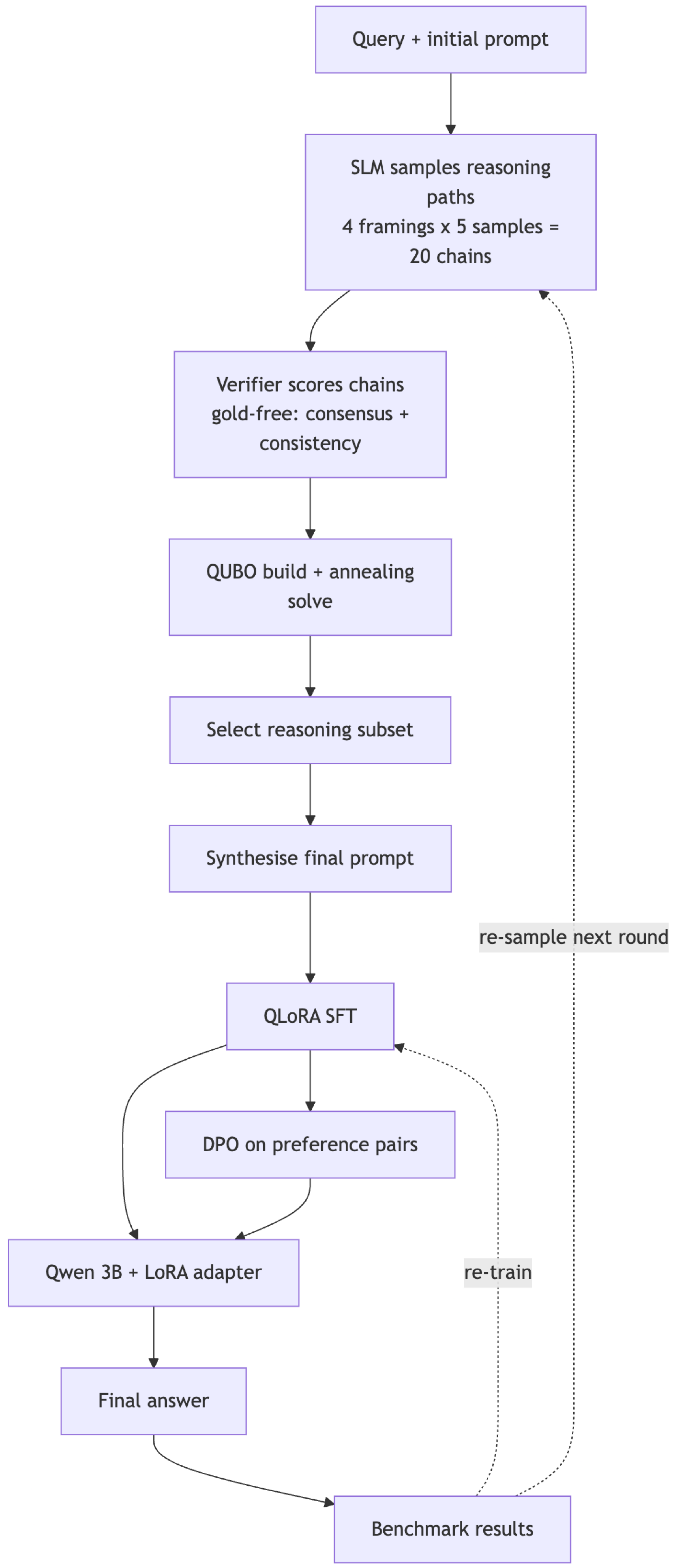
LoRa Configuration

Hyperparameter	Value	Role
LoRA rank (r)	16	Capacity of adapter
LoRA alpha	32	Scaling (effective LR multiplier)
LoRA dropout	0.05	Reduces overfitting
Target modules	`q_proj`, `v_proj`, `k_proj`, `o_proj`	All attention projections
Bias	none	Standard LoRA setup
Task type	CAUSAL_LM	Next-token prediction

Output comparison

Metric / Evaluation	Before	After
Pipeline, 50q	56%	(after run was 25q)
Pipeline, same 25q	52%	76%

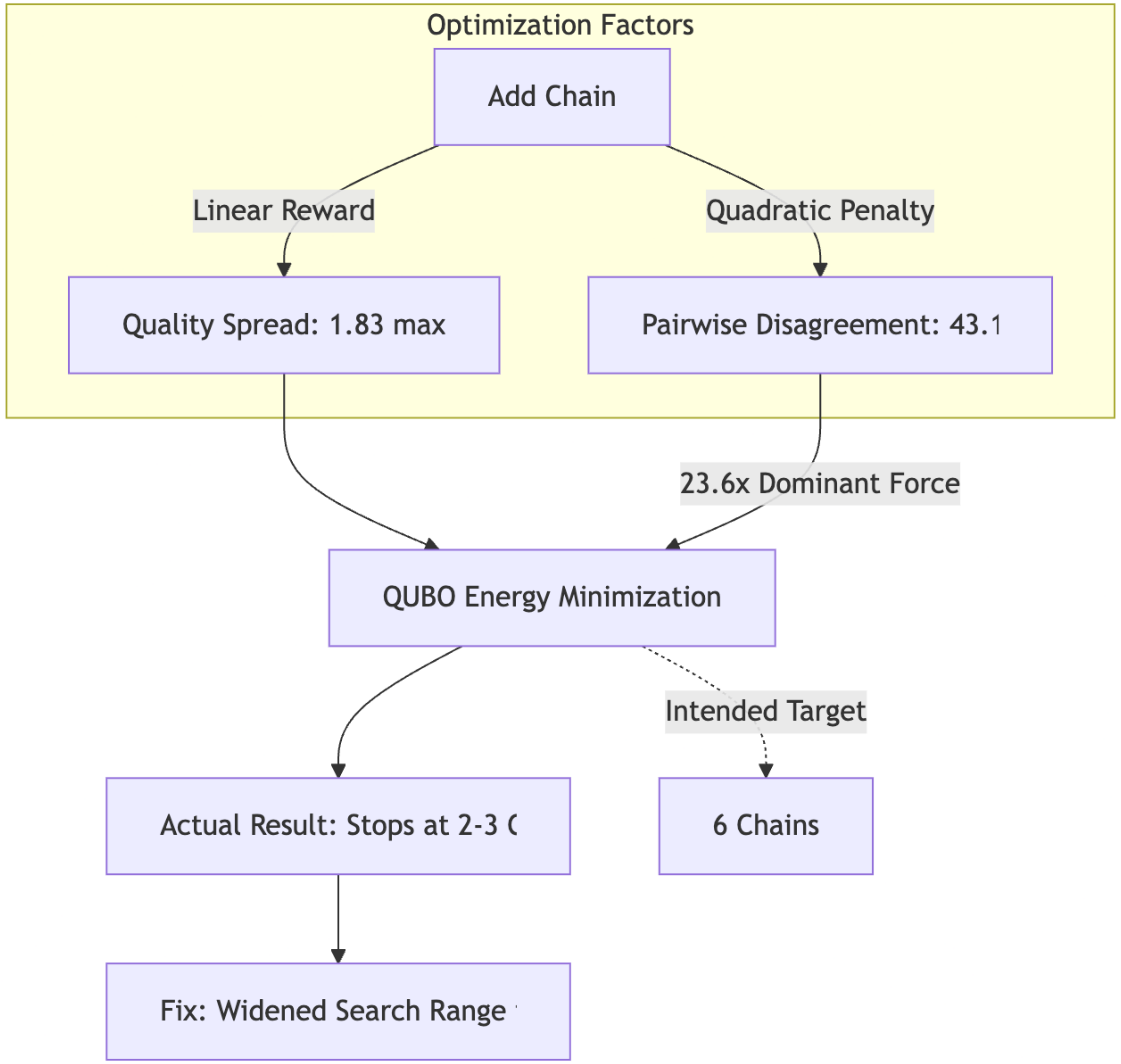
Hyper-parameter fine tuning

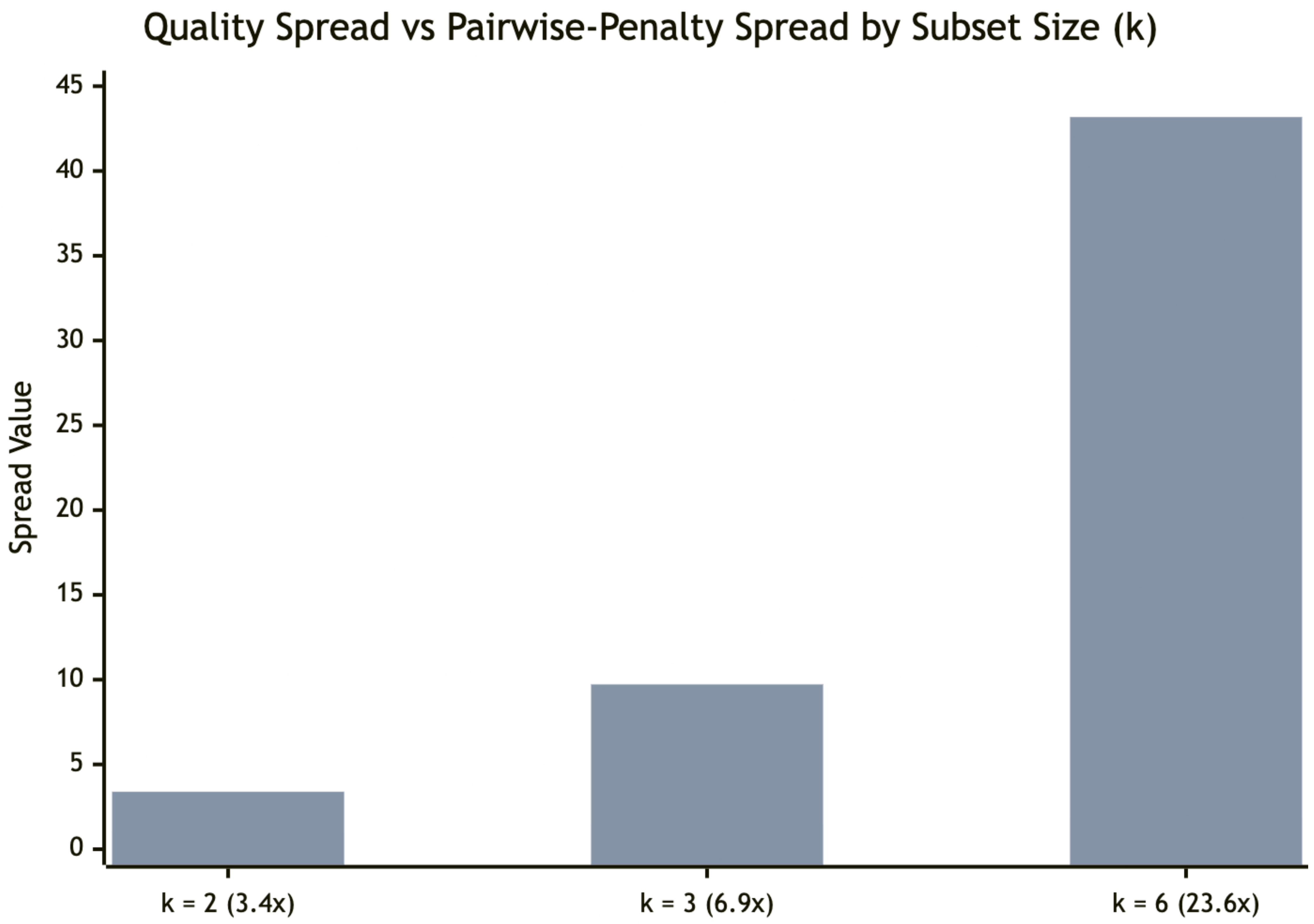


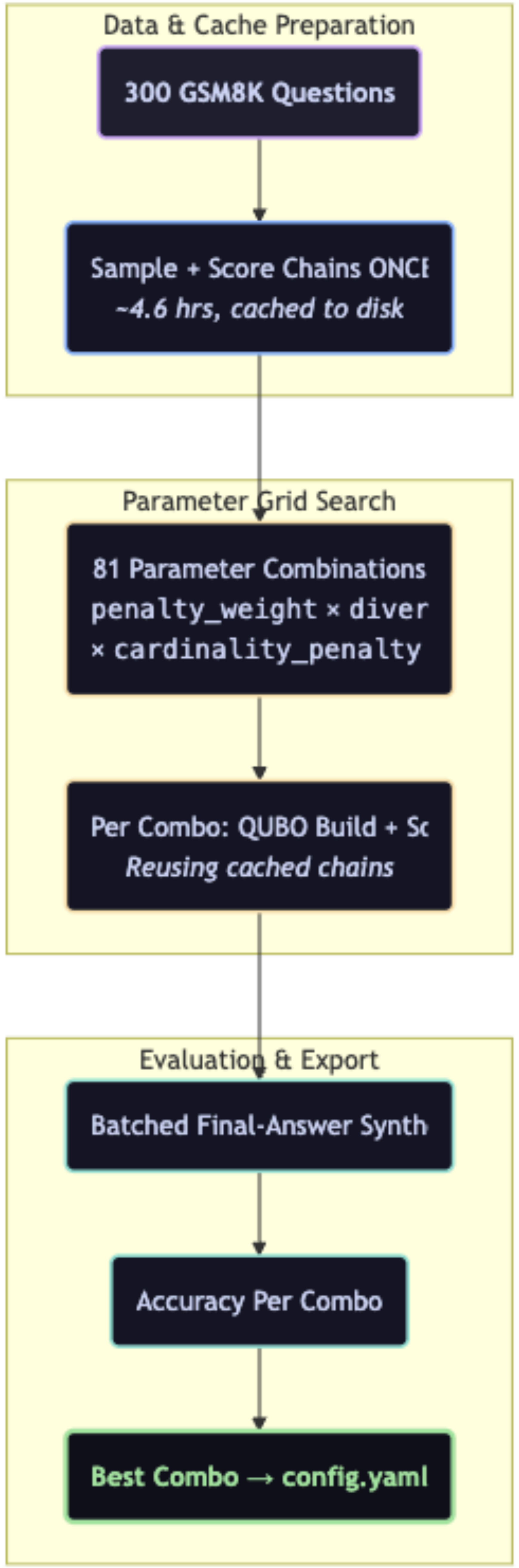
Bugs Quietly Inflating Benchmark Metrics

- **MCQ Extraction Fix:** Stopped matching bare letters inside words (e.g., *"triangle"* → Option A). Updated parser to abstain rather than guess.
- **Strict Answer Boundaries:** Fixed substring matching so flawed reasoning (e.g., *"... = False, therefore True"*) isn't falsely credited for containing *"False"*.
- **Dataset Retyping:** Reclassified BIG-bench Hard (BBH) to commonsense scoring, as 63% of ground truths are MCQ options.
- **Data Leak Cleanup:** Removed test-set contamination from MMLU training data; sanitized StrategyQA, ARC, and MATH training splits against evaluation sets

Why QUBO Only Selected 2 of 6 Chains






Data & Cache Preparation
300 GSM8K Questions
Sample + Score Chains ONCI
~4.6 hrs, cached to disk
Parameter Grid Search
81 Parameter Combinations
penalty_weight × diver
× cardinality_penalty
Per Combo: QUBO Build + Sc
Reusing cached chains
Evaluation & Export
Batched Final-Answer Synth
Accuracy Per Combo
Best Combo → config.yaml